

Strukturni dizajn paterni

U ovom dokumentu je objašnjena primjena strukturnih dizajn paterna na sistem pametnog parkinga. Cilj dodavanja paterna nije bio samo formalno ubaciti nove klase u dijagram, nego pokazati gdje u sistemu stvarno postoji potreba za boljom organizacijom klasa i smanjenjem zavisnosti između njih.

Na osnovu postojećeg klasnog dijagrama, u sistem su dodana dva strukturna paterna:

- Adapter
- Facade
- Proxy

Na novom dijagramu su dodani elementi `IPlatniServis`, `BankovniSistemAdapter`, `EksterniPlatniSistem` i `RezervacijaFacade`. Ovi elementi su povezani sa već postojećim klasama kao što su `Rezervacija`, `ParkingMjesto`, `Placanje`, `EmailObavijest`, `Cjenovnik` i kontrolerske klase.

1. Adapter

Adapter je strukturni dizajn patern koji se koristi kada dvije klase ili dva sistema ne mogu direktno komunicirati jer imaju različite interfejse. Adapter tada služi kao posrednik koji prilagođava jedan interfejs drugom.

U našem sistemu pametnog parkinga Adapter se koristi kod plaćanja rezervacije. Sistem ima klasu `Placanje`, ali stvarna obrada plaćanja može zavisiti od vanjskog platnog sistema, npr. banke ili nekog servisa za kartično plaćanje.

Problem je u tome što vanjski platni sistem ne mora imati iste metode kao naš sistem. Na primjer, naš sistem može očekivati metodu:

```
izvrsiPlacanje(placanje: Placanje): bool
```

dok vanjski sistem može imati metodu:

```
obradiTransakciju(iznos: double, brojKartice: String): bool
```

Zbog toga nije dobro da ostatak aplikacije direktno zavisi od vanjskog platnog sistema. Ako bi se kasnije promijenila banka ili način plaćanja, morale bi se mijenjati klase koje koriste plaćanje. Da bi se to izbjeglo, uveden je Adapter patern.

Za Adapter patern su dodani:

- IPlatniServis
- BankovniSistemAdapter
- EksterniPlatniSistem

IPlatniServis predstavlja interfejs koji definiše metode koje sistem koristi za plaćanje:

+izvrsiPlacanje(placanje: Placanje): bool

+pokreniPovratNovca(placanje: Placanje): bool

BankovniSistemAdapter implementira taj interfejs i prilagođava pozive našem eksternom platnom sistemu. U sebi ima referencu na EksterniPlatniSistem, preko kojeg se stvarno obrađuje transakcija.

EksterniPlatniSistem predstavlja vanjski sistem koji ima svoje metode:

+obradiTransakciju(iznos: double, brojKartice: String): bool

+refundirajTransakciju(transakcijskiBroj: String): bool

Kada korisnik plaća rezervaciju, sistem ne treba direktno pozivati vanjski platni sistem. Umjesto toga koristi se IPlatniServis, a konkretni adapter (BankovniSistemAdapter) preuzima zadatak da taj poziv prilagodi vanjskom sistemu.

Na ovaj način naš sistem ostaje nezavisan od konkretne implementacije platnog servisa. Ako se kasnije uvede drugi platni servis, može se dodati novi adapter, bez velikih izmjena u ostatku sistema.

2. Facade

Facade je strukturni dizajn patern koji se koristi kada imamo složen proces koji uključuje više klasa. Umjesto da kontroler ili neka druga klasa direktno poziva veliki broj drugih klasa, uvodi se jedna posebna klasa koja predstavlja jednostavan ulaz u taj proces.

U našem sistemu Facade se koristi za proces rezervacije parking mjesta. Rezervacija nije samo jednostavno kreiranje jednog objekta. Ona uključuje više koraka, kao što su:

- provjera da li je parking mjesto dostupno
- obračun cijene rezervacije
- kreiranje rezervacije
- plaćanje rezervacije
- slanje email obavijesti korisniku

Bez Facade paterna, RezervacijaController bi morao direktno komunicirati sa klasama Rezervacija, ParkingMjesto, Cjenovnik, Placanje i EmailObavijest. To bi kontroler učinilo previše složenim i nepreglednim.

Zbog toga je dodana klasa:

RezervacijaFacade

Klasa RezervacijaFacade

RezervacijaFacade sadrži metode:

+kreirajKompletnuRezervaciju(rezervacija: Rezervacija, placanje: Placanje): bool

+izmijeniRezervaciju(rezervacija: Rezervacija): bool

+produziRezervaciju(rezervacijald: int, noviKraj: DateTime): bool

+otkaziRezervaciju(rezervacijald: int): bool

Ova klasa objedinjuje logiku koja je vezana za rezervaciju. Na primjer, kod kreiranja kompletne rezervacije ona može provjeriti dostupnost parking mjesta, izračunati cijenu, kreirati rezervaciju, pokrenuti plaćanje i pripremiti email obavijest.

RezervacijaController ne mora direktno pozivati sve klase koje učestvuju u procesu rezervacije. Umjesto toga, kontroler poziva RezervacijaFacade, a facade dalje koordinira ostale klase.

To znači da se složenost procesa sakriva iza jedne klase. Kontroler ostaje jednostavniji, a logika rezervacije je organizovana na jednom mjestu.

Korištenjem Facade paterna dobija se:

- jednostavniji i pregledniji RezervacijaController
- jasnije organizovan proces rezervacije
- manje direktnih zavisnosti između kontrolera i model klasa
- lakše održavanje sistema
- lakše proširenje procesa rezervacije u budućnosti

Na primjer, ako se kasnije promijeni način obračuna cijene ili način slanja email obavijesti, te promjene se mogu napraviti u RezervacijaFacade, bez velikih izmjena u kontroleru.

3. Proxy

Proxy patern je dodan kod plaćanja kako bi se prije same transakcije provjerilo da li korisnik ima pravo izvršiti plaćanje. U našem sistemu korisnik ne bi trebao moći platiti rezervaciju koja nije njegova ili koja nije u statusu u kojem je plaćanje dozvoljeno.

Zbog toga je uvedena klasa PlacanjeProxy. Ona se koristi kao posrednik između kontrolera za plaćanje i same klase Placanje. Kada korisnik pokrene plaćanje, zahtjev prvo prolazi kroz PlacanjeProxy, gdje se provjerava da li korisnik ima pravo nastaviti proces. Ako je provjera uspješna, plaćanje se proslijeđuje klasi Placanje; ako nije, plaćanje se ne izvršava.

Na ovaj način se uvodi dodatna sigurnost u proces plaćanja, a PlacanjeController ostaje jednostavniji jer ne mora direktno sadržavati svu logiku provjere prava pristupa. Klasa PlacanjeProxy sadrži metode za provjeru prava plaćanja i izvršavanje plaćanja nakon uspješne provjere.

Zaključak

U sistem pametnog parkinga dodana su dva strukturna dizajn paternna: Adapter i Facade.

Adapter je dodan u dijelu sistema koji se odnosi na plaćanje. Njegova uloga je da omogućiti komunikaciju između našeg sistema i eksternog platnog sistema bez direktne zavisnosti od konkretne implementacije tog vanjskog sistema.

Facade je dodan u dijelu sistema koji se odnosi na rezervacije. Njegova uloga je da pojednostavi složen proces rezervacije parking mjesta i da sakrije detalje rada više klasa iza jedne klase RezervacijaFacade.

Ovi paterni su dodani zato što imaju stvarnu ulogu u sistemu. Adapter pomaže kod integracije sa vanjskim servisom za plaćanje, dok Facade olakšava upravljanje složenim procesom rezervacije. Time sistem postaje pregledniji, fleksibilniji i jednostavniji za održavanje.